

Rekurencja (inaczej: rekursja) i jej usuwanie (derekursywacja)

procedura (funkcja, podprogram itd.), która wywołuje samą siebie; wywołanie to może być niebezpośrednie.

Kwestia poprawności:

Własność stopu osiągnięta przez:

- parametr wywołania, np. malejący w kolejnych wywołaniach, gdy osiąga graniczną wartość to nie ma dalszych wywołań; np. rekurencyjne obliczanie $n!$;
- ograniczenia wielkości przetwarzanej struktury, której elementy są odpowiednio znakowane w kolejnych wywołaniach rekurencyjnych, np. w rekurencyjnej (czyli standardowej) wersji algorytmu DFS.

Reguły konstrukcji procedur rekurencyjnych:

- musi istnieć przypadek graniczny (baza rekurencji) rozwiązywalny bez rekurencji;
- rekurencyjne wywołania muszą dążyć do przypadku granicznego;
- w dowodzie poprawności należy założyć poprawność wywołań rekurencyjnych (indukcja);
- należy unikać ponownych wywołań rekurencyjnych wykonujących te same obliczenia (czyli dla takich samych wartości parametrów);

Jak wykonywana jest procedura "oblicz(n)" ?

```
procedure oblicz(n) ;  
var x;  
    if n=0 then return;  
    ...  
    oblicz(n-1) ;  
    x := x + 2;  
    ...  
    return;  
end.
```

- wywołanie rekurencyjne procedury oblicz() powoduje fizyczny skok (goto, JUMP) do początku procedury;
- po wykonaniu tego skoku obiekty n, x to nowe obiekty, inne niż przed skokiem;
- po wykonaniu return w wywołaniu zagnieżdżonym sterowanie musi wrócić do instrukcji następnej po wywołaniu, czyli do $x := x + 2$;
- po powrocie do tej instrukcji obiekty x, n z wywołania zagnieżdżonego (właśnie zakończonego) są zapominane, a przywracane są odpowiednie obiekty sprzed wywołania rekurencyjnego.

Wniosek:

- obsługa rekurencji, przed wywołaniem procedury musi zapamiętać aktualny kontekst i adres do którego należy wrócić po zakończeniu wywołania rekurencyjnego (rekord aktywacyjny);
- wywołania się nakładają, więc przywracanie kontekstu musi się odbywać na zasadzie ostatni zapamiętany = pierwszy przywracany;
- czyli z użyciem stosu.

Rekordy aktywacyjne odkładane na stos mogą być dość duże, gdy brak jest monitorowania wielkości stosu podczas wykonania łatwo o przepełnienie i błąd wykonania.

Rekurencja ogonowa

gdy wywołanie podprogramu jest ostatnią wykonywaną instrukcją kontekst nie musi być pamiętany.

```
PrintList(p)
// p - wskaźnik do pierwszego na liście
    if p=NULL then return;
    print(info(p));
    PrintList(next(p));
end.
```

Rekurencję ogonową traktuje się jako raczej niezręczne (pogarszające efektywność kodu) użycie rekurencji.

Rekurencja ogonowa jest łatwa do wyeliminowania, optymalizację taką mogą wykonywać kompilatory.

Eliminując rekurencję ogonową najprościej zamknąć kod w nieskończonej pętli WHILE:

```
PrintList (p)
// p - wskaźnik do pierwszego na liście
    while TRUE do
        if p=NULL then return;
        print (info(p));
        p := next(p);           // zmiana kontekstu
        // idź na początek, czyli kolejna iteracja
end.
```

Przykład (nieco) bardziej skomplikowany:

```
BinSearch(x, A, L, R)
// w tablicy A[L..R], wersja "2 porównania";
// zwraca dowolne  $M \in L..R$  takie że  $A[M]=x$ ,
// NULL jeśli  $x \notin A[L..R]$ 

    if  $L > R$  return NULL;

     $M := (L+R)/2$ ;
    if  $x=A[M]$  then return M;

    if  $x > A[M]$  then
        return BinSearch(x, A, M+1, R);
    else
        return BinSearch(x, A, L, M-1);
end.
```

Oba wywołania są ogonowe – o tym decyduje kolejność wykonywania, a nie pozycja w tekście programu.

Ćwiczenie: usuń obie rekurencje ogonowe (tzn. przekształć algorytm na równoważny, nie zawierający wywołań rekurencyjnych).

Wieże Hanoi

Na wieży (patyku) A jest n krążków o różnych średnicach, uporządkowanych malejąco w górę wieży.

Zadanie:

- przenieść n krążków z wieży A na wieżę B,
- wykorzystując jako pomocniczą wieżę C,
- pojedynczo, nie kładąc większego na mniejszym.

```
Towers(n, A, B, C);  
  if n=0 then return;  
  Towers(n-1, A, C, B);  
  A -> B;           // przenieś jeden krążek z A na B  
  Towers(n-1, C, B, A);  
end.
```

Poprawność: indukcja, łatwa.

Złożoność (liczba ruchów):

$$T(0) = 0$$

$$T(n) = 2 T(n-1) + 1, \text{ dla } n \geq 1$$

odgadnąć: $T(n) = 2^n - 1, n \geq 0$, dowód indukcją łatwy.

Eliminacja rekurencji ogonowej:

z prostą modyfikacją – test zakończenia wstawiliśmy do warunku pętli:

```
Towers2(n, A, B, C);  
  while n>0 do  
    Towers2(n-1, A, C, B);  
    A -> B;  
    n--;  
    swap(C,A);           // zamień role A i C  
end.
```

Usuwanie rekurencji – metoda ogólna

Tak działa kompilator tłumacząc program zawierający wywołania funkcji na kod w języku wewnętrznym, np. w języku asemblera.

Terminologia:

- Kontekst =
parametry aktualne, obiekty (stałe i zmienne) lokalne;
- Rekord aktywacji (ramka) =
kontekst + adres powrotu;
- Stos =
stos rekordów aktywacji dla wykonań funkcji, które jeszcze się nie zakończyły (czyli do których jeszcze wrócimy).

Wywołanie funkcji:

- wstaw na stos rekord aktywacji zawierający aktualny kontekst + adres powrotu;
- wygeneruj kontekst dla nowego (rekurencyjnego) wywołania;
- aktualny licznik rozkazów := początek kodu funkcji;

Zakończenie procedury:

- pobierz rekord aktywacji ze stosu;
- zastąp istniejący kontekst zawartością z pobranego rekordu;
- skocz do adresu powrotu zawartego w pobranym rekordzie aktywacji.

Przykład: wieże Hanoi

```
Towers(n, A, B, C)
// ramka = {n, A, B, C, adres_powrotu}
1:  if n=0 then goto 5;
2:  push[n,A,B,C,3];
    n--;
    swap(C,B);
    goto 1;
3:  A -> B;
4:  push[n,A,B,C,5];
    n--;
    swap(A,C);
    goto 1;
5:  if stackempty() then return;
    [n,A,B,C,adres_powrotu] := pop(); // cała ramka
    goto adres_powrotu;
end.
```

<pre>Towers(n, A, B, C); 1. if n=0 then return; 2. Towers(n-1, A, C, B); 3. A -> B; 4. Towers(n-1, C, B, A); 5. end.</pre>

Jeśli programujemy w języku wysokiego poziomu (np. C++) i zależy nam na tym aby nie używać skoku bezwarunkowego (tak się zaleca), to możemy kod strukturyzować za pomocą pętli WHILE.

Metoda:

- fragmenty kodu, do których wykonywane są skoki zamykamy w blokach - częściach kodu wewnątrz WHILE;
- o tym który fragment wykonywany jest w danej iteracji WHILE, rozstrzygamy odpowiednimi instrukcjami IF THEN ELSE lub SWITCH CASE.

Zauważmy: etykiety (adresy) 2 i 4 nie są potrzebne - nie ma do nich skoku z innego miejsca.

```

Towers(n, A, B, C)
    // kontekst = zmienne n, A, B, C, adres
    adres := 1;
    while TRUE do
        if adres=1 then
            if n=0 then
                adres := 5;
            else
                push[n,A,B,C,3];
                swap(C, B);
                n--;
                adres := 1;
            else if adres=3 then
                A -> B;
                push[n,A,B,C,5];
                swap(A, C);
                n--;
                adres := 1;
            else
                // adres=5
                if stackempty() then return;
                [n,A,B,C,adres] := pop();
            end if;
        end while;
    end Towers.

```

UWAGA (MOCNA SUGESTIA)

Zamieniając rekurencję na iterację nie używającą GOTO, łatwiej jest najpierw skonstruować wersję z GOTO, a potem ją zamienić na wersję z pętlą WHILE, bez GOTO.

Przykład 2

pokażemy, że wynik mechanicznego usunięcia rekursji w algorytmie Quicksort

```
quicksort (l, r)
  if l<r then
    m := partition(l,r);
    quicksort(l,m-1);
    quicksort(m+1,r);
end.
```

można przekształcić w KLASYCZNY QUICKSORT bez rekurencji, ze stosem:

```
quicksort(l,r)
  while l<r OR NOT stackempty() do
    if l<r then
      m := partition(l,r);
      push(m+1); push(r);
      r := m-1;
    else
      r := pop(); l := pop();
    end.
end.
```

Najpierw mechanicznie, zgodnie z ogólnym algorytmem usuwania rekursji, zamiast każdego wywołania rekurencyjnego:

- wstawiamy aktualny kontekst na stos;
- konstruujemy nowy kontekst;
- skaczemy na początek,

a na końcu ściągamy ze stosu poprzedni kontekst i skaczemy pod podany w tym kontekście adres.

```
quicksort (l, r)
  if l<r then
    m := partition(l,r);
    quicksort(l,m-1);
    quicksort(m+1,r);
end.
```

```
quicksort(l,r)
  0: if l>=r then goto 3;
    m := partition(l,r);
  1: push(l); push(m); push(r); push(2);
    r := m-1; goto 0;
  2: push(l); push(m); push(r); push(3);
    l := m+1; goto 0;
  3: if stackempty() return;
    addr:=pop(); r:=pop(); m:=pop(); l:=pop();
    goto addr;
end.
```

Teraz stopniowo optymalizujemy kod. Zauważmy:

- etykieta 1 jest zbędna, bo nie ma do niej skoku;
- zapamiętanie kontekstu w etykiecie 2 zbędne, bo po powrocie do tego kontekstu idziemy do 3 i od razu ściągamy następny (starszy) kontekst; (rekurencja w linii 2 jest ogonowa!)
- skok do etykiety 3 odbywa się tylko poprzez zapamiętanie w instrukcji 2 ramki z adresem powrotu 3;
- zatem adres 3 jako adres powrotu staje się zbędny, więc adres powrotu równy 2 staje się domyślny i nie trzeba go pamiętać na stosie w linii 1;
- w linii 1 nie trzeba zapamiętywać na stosie parametru l, bo po ściągnięciu kontekstu ze stosu (odbywa się ono pod etykietą 3) jest skok do etykiety 2 i wartość l się zmienia;

Po uproszczeniu, stara i nowa wersja:

```
quicksort(l,r)
  0: if l>=r then goto 3;
      m := partition(l,r);
  1: push(l); push(m); push(r); push(2);
      r := m-1; goto 0;
  2: push(l); push(m); push(r); push(3);
      l := m+1; goto 0;
  3: if stackempty() return;
      addr:=pop(); r:=pop(); m:=pop(); l:=pop();
      goto addr;
end.
```

```
quicksort(l,r)
  0: if l>=r goto 3;
      m := partition(l,r);
      push(m); push(r);
      r := m-1; goto 0;
  2: l := m+1; goto 0;
  3: if stackempty() then return;
      r := pop(); m := pop(); goto 2;
end.
```

Można usunąć linię z etykietą 2, jeśli

- na stos położymy m+1 zamiast m i dzięki temu, w linii 3 ściągniemy m+1 ze stosu od razu pod l,
- oraz w linii 3, zamiast skoku do 2 i dalej do 0, od razu damy skok do 0.

Po uproszczeniu, stara i nowa wersja:

```
quicksort(l,r)
  0: if l>=r goto 3;
    m := partition(l,r);
    push(m); push(r);
    r := m-1; goto 0;
  2: l := m+1; goto 0;
  3: if stackempty() then return;
    r := pop(); m := pop(); goto 2;
end.
```

```
quicksort(l,r)
0:  if l>=r then go to 3;
    m := partition(l,r);
    push(m+1); push(r);
    r := m-1; goto 0;
3:  if stackempty() then return;
    r := pop(); l := pop(); goto 0;
end.
```

teraz "while" zamiast "goto ":

```
quicksort(l,r)
  while true do
    while l<r do
      m := partition (l,r);
      push(m+1); push(r);
      r := m-1;
    if stackempty() then break;
    r := pop(); l := pop();
  end.
```

Wstawiamy testowanie stosu do warunku pętli i otrzymujemy

KLASYCZNY QUICKSORT bez rekurencji, ze stosem:

```
quicksort(l,r)
  while l<r OR NOT stackempty() do
    if l<r then
      m := partition(l,r);
      push(m+1); push(r);
      // prawe podzadanie zapamietaj na stosie
      r := m-1; // przejdź do lewego podzadania
    else
      r := pop(); l := pop();
  end.
```

Przypomnienie kwestii pamięci na stos w algorytmie Quicksort:

Niezbędne jest uzupełnienie (dla wersji z rekurencją i bez) redukujące stos do pesymistycznej wielkości $O(\log n)$:

- porównuj rozmiary podzadań otrzymywanych w wyniku partition();
- iteracyjnie przechodź do mniejszego podzadania, większe zaś zapamiętuj na stosie.

Rekurencja w funkcjach zwracających wartość

- dodatkowy stos i operacje:
 - pushf (item),
 - item := popf();
- powrót z podprogramu zawiera:
 - pushf(obliczona_wartość);
- wykorzystanie wartości zwracanej przez funkcję
 - zastępowane przez popf().

```
Fib(n)
  // po prostu Fibonacchi
  // nigdy nie stosuj rekurencji
  // do obliczania liczb Fibonacciego !!!
  if n=0 then return 0
  else
    if n=1 then return 1;
    return Fib(n-1) + Fib(n-2);
end.
```

Wstawiamy adresy w programie i dodatkowe zmienne.

```
Fib(n)
0:  if n=0 then return 0;
    else
      if n=1 then return 1;
1:  f1 := Fib(n-1);
2:  f2 := Fib(n-2);
3:  return f1 + f2;
4:
end.
```

Stosujemy reguły ogólne oraz niezależnie, dodatkowe instrukcje związane z faktem zwracania wartości przez funkcję.

```
Fib(n)
s:  if n=0 then
      pushf(0); go to 4;
    else
      if n=1 then
        pushf(1); go to 4;
1:  push(n); push(2); n = n-1; go to 0;
2:  push(n); push(3); n = n-2; go to 0;
3:  f2 := popf(); f1 := popf(); pushf(f1+f2);
    // f1,f2 chwilowe, nie należą do ramki-kontekstu
    // zamiast nich można: pushf(popf()+popf());
4:  if (stackempty()) then go to 5;
    addr := pop(); n := pop(); go to addr;
5:  return popf();
end.
```

Dowód poprawności indukcją na n .

Wykazać, że jeśli

- obliczenie jest w linii 0,
- $n \geq 0$,
- stos główny ma postać (zawartość) S ,
- stos funkcyjny ma postać (zawartość) S_f ,

to po pewnym czasie obliczenie dojdzie do linii 5 z:

- niezmienną zawartością stosu głównego S ,
- stosem funkcyjnym powiększonym o wartość liczby Fibonacciego F_n .

Baza indukcji: $n=0$, $n=1$.

Krok indukcyjny: analiza programu.

QED